

Formal Verification Report: Morpho Blue

- Repository: <https://github.com/morpho-org/morpho-blue>
 - Latest Commit Hash: [57d444d](#)
 - Date: March 2026
 - Author: [@alexzoid](#)
 - Certora Prover version: 8.8.1
-

Table of Contents

1. [About Morpho Blue](#)
2. [Formal Verification Methodology](#)
 - [Verification Approach](#)
 - [Types of Properties](#)
 - [Verification Process](#)
 - [Project Structure](#)
 - [Assumptions](#)
3. [Verification Properties](#)
 - [High-Level](#)
 - [Valid State](#)
 - [Variable Transitions](#)
 - [State Transitions](#)
 - [Unit Tests](#)
 - [Accrue Interest](#)
 - [Reachability](#)
 - [Reverts](#)
 - [Access Control](#)
4. [Mutation Testing](#)
 - [What is Mutation Testing](#)
 - [liquidityInvariant](#)
 - [supplySharesSolvency](#)
 - [borrowSharesConservation](#)
5. [Setup and Execution](#)
 - [Common Setup \(Steps 1-5\)](#)
 - [Remote Execution](#)
 - [Local Execution](#)
 - [Running Verification](#)

- [Running Mutation Testing](#)

About Morpho Blue

Morpho Blue is a noncustodial lending protocol for the Ethereum Virtual Machine that provides permissionless market creation with isolated lending pools. Each market is defined by a unique tuple of (loanToken, collateralToken, oracle, IRM, LLTV) and operates independently. The protocol is a singleton immutable contract with no governance over individual markets -- the owner can only enable new IRMs and LLTVs, set fees, and transfer ownership.

The entire protocol lives in a single contract `Morpho.sol` (555 lines). It uses no proxy patterns, is not upgradeable, and has no Diamond facets. All logic is in the singleton contract, delegating only to internal pure/view libraries for math operations: `MathLib` (WAD-scaled arithmetic), `SharesMathLib` (virtual-shares conversion), `UtilsLib` (helpers), `SafeTransferLib` (safe ERC20 wrappers), and `MarketParamsLib` (market ID computation).

Core operations include supply/withdraw (lending), borrow/repay (borrowing), collateral management, liquidation with bad debt socialization, unrestricted flash loans, and EIP-712 authorization delegation. Interest accrual is lazy, triggered at the start of most operations. Virtual shares (1e6) mitigate inflation attacks on empty markets.

Formal Verification Methodology

Certora Formal Verification (FV) provides mathematical proofs of smart contract correctness by verifying code against a formal specification. Unlike testing and fuzzing which examine specific execution paths, Certora FV examines all possible states and execution paths.

The process involves crafting properties in CVL (Certora Verification Language) and submitting them alongside compiled Solidity smart contracts to a remote prover. The prover transforms the contract bytecode and rules into a mathematical model and determines the validity of rules.

Verification Approach

The valid-state surface is verified under two complementary market regimes — a single-market narrowing and an unrestricted per-id form — to keep the SMT problem tractable while still covering the cross-id reasoning that the single-market form cannot express. The two regimes share the same shared base

([setup/morpho.spec](#) — methods block, constants, non-id-keyed ghosts) and the same harness ([MorphoHarness](#)); they differ only in how the per-market ghost surface is shaped.

- **One-market regime** ([morpho_valid_state_one.spec](#) + [setup/morpho_one.spec](#)) — the per-market ghost surface is collapsed to scalar (no-id) ghosts. Sload mirror checks on the `_Morpho.market[KEY id].*`, `_Morpho.position[KEY id][KEY user].*`, and `_Morpho.idToMarketParams[KEY id].*` slots force single-id consistency: any path that touches a second distinct id with a different stored value is infeasible because the scalar ghost cannot simultaneously equal both values. Every per-market valid-state invariant proves in this regime; because markets do not interfere, the proof generalises to every market.
- **Many-market regime** ([morpho_valid_state_many.spec](#) + [setup/morpho_many.spec](#)) — per-id ghosts are retained and the full valid-state invariant set is lifted to `forall MorphoHarness.Id id.`. This is the regime used by every non-valid-state spec category (high-level, state transitions, unit tests, reverts, reachability, access control, accrue interest, variable transitions), since their rules already fix or quantify over `id` directly.

The non-valid-state spec categories import [setup/morpho_many.spec](#) for their per-id ghost surface. Only the valid-state property set is duplicated across the two regimes (every invariant proven once in each); all 23 valid-state invariants verify under both confs.

Types of Properties

Properties are categorized following the [official Certora methodology](#). Valid State, Variable Transitions and State Transitions properties are **parametric** -- they are automatically verified against every external function in the contract, including functions added after the specification is completed. High-Level properties target specific function sequences.

High-Level -- Complex multi-step properties verifying business logic integrity. Unlike parametric properties, these target specific function sequences to validate end-to-end protocol behavior.

Valid State -- System-wide invariants that MUST always hold true. These properties define the fundamental constraints of the protocol, such as accounting consistency and structural integrity. Once proven, these invariants serve as trusted assumptions in other properties via `requireInvariant`, reducing verification complexity.

Variable Transitions -- Properties that verify specific storage variables change only under expected conditions. The process captures a variable value before instruction execution, runs the instruction, then asserts the variable changed only as permitted or remained unchanged.

State Transitions -- Properties that verify the correctness of transitions between valid states. Building upon the valid state invariants, these properties ensure the protocol's state machine operates correctly and that state changes are both authorized and sequentially valid.

Unit Tests -- Properties that verify basic behavior of individual functions: direct effects on state, and non-effects on unrelated state. Unlike parametric properties, each unit test targets a specific function call.

Accrue Interest -- Properties verifying that interest accrual is correctly triggered across lending operations. Each rule uses `lastStorage` to compare two execution paths: one with an explicit `accrueInterest()` call before the operation, and one without. Identity of final storage proves the operation internally accrues interest.

Reachability -- Properties using `satisfy()` to prove that key scenarios are reachable, confirming non-vacuous verification. Basic function reachability confirms each function can execute successfully; conditional reachability validates that specific states and paths are achievable.

Reverts -- Properties verifying revert conditions using `@withrevert` and `lastReverted`. These confirm that functions correctly reject invalid inputs, enforce preconditions, and revert under expected circumstances.

Access Control -- Parametric rules verifying that unauthorized callers cannot modify protected state. These confirm that role-gated functions reject calls from non-privileged addresses.

Verification Process

1. **Setup phase:** Define ghost variables, storage hooks, and helper definitions to model contract state in CVL. Establish the verification harness and configuration. This phase addresses several prover limitations:
 - Math library summaries ([morpho_math_lib.spec](#)): Internal math functions (`wMulDown`, `mulDivDown`, `wTaylorCompounded`) are summarized in CVL to avoid prover timeouts from complex WAD arithmetic.
 - Shares math library summaries ([morpho_shares_math_lib.spec](#)): Share-to-asset conversion functions (`toSharesDown`, `toSharesUp`, `toAssetsDown`, `toAssetsUp`) are summarized in CVL for tractability.
 - Safe transfer library summaries ([solmate_safe_transfer_lib.spec](#)): SafeTransferLib's inline assembly is summarized in CVL to model ERC20 token transfers.
 - ERC20 model ([erc20.spec](#)): Full CVL model for ERC20 tokens with bounded account sets, ghost-tracked balances, and sum invariants.
 - Environment constraints ([env.spec](#)): Standard environment assumptions (no ETH, non-zero sender, realistic timestamps).
2. **Crafting Properties:** Write invariants and rules in CVL, starting with valid state invariants (which become trusted preconditions for other rules), then state/variable transitions, and finally high-level properties.

Project Structure

```
certora/
├── confs/                                # Prover configuration files
│   ├── access_control.conf
│   ├── accrue_interest.conf
│   ├── high_level.conf
│   ├── high_level_mutations_borrowSharesConservation.conf
│   ├── reachability.conf
│   ├── reverts.conf
│   ├── state_transitions.conf
│   ├── unit_tests.conf
│   ├── valid_state_one.conf              # One-market regime (scalar ghosts)
│   ├── valid_state_many.conf            # Many-market regime (per-id ghosts)
│   ├── valid_state_mutations_liquidityInvariant.conf
│   ├── valid_state_mutations_supplySharesSolvency.conf
│   ├── variable_transitions.conf
│   └── debug/                            # Debug/sanity configurations
│       ├── erc20_max_users.conf
│       ├── morpho_valid_state_debug.conf
│       └── sanity.conf
├── harnesses/                            # Verification harnesses
│   ├── HelperCVL.sol
│   └── MorphoHarness.sol
├── mutations/                            # Mutation testing files
│   ├── add_mutation.sh
│   ├── borrowSharesConservation/         # 10 manual mutants
│   │   ├── 1.sol ... 10.sol
│   ├── liquidityInvariant/              # 13 manual mutants
│   │   ├── 1.sol ... 13.sol
│   └── supplySharesSolvency/            # 10 manual mutants
│       ├── 1.sol ... 10.sol
├── specs/                                # CVL specification files
│   ├── morpho_access_control.spec        # Access control rules
│   ├── morpho_accrue_interest.spec       # Interest accrual idempotency
│   ├── morpho_high_level.spec            # High-level behavioral rules
│   ├── morpho_reachability.spec          # Reachability rules
│   ├── morpho_reverts.spec               # Revert condition rules
│   ├── morpho_state_transitions.spec     # State transition rules
│   ├── morpho_unit_tests.spec            # Unit test rules
│   ├── morpho_valid_state_one.spec       # Valid state invariants (one-market
regime)
│   ├── morpho_valid_state_many.spec      # Valid state invariants (many-market
regime)
│   ├── morpho_variable_transitions.spec  # Variable transition rules
│   ├── setup/                            # Setup specs (ghosts, hooks, summaries)
│   │   └── morpho.spec                   # Shared base: methods, constants, non-
id-keyed ghosts
```

```

|   |— morpho_one.spec                    # One-market regime: scalar (no-id)
ghosts
|   |— morpho_many.spec                  # Many-market regime: per-id ghosts
|   |— libs/                             # Library summaries and models
|       |— env.spec                     # Environment constraints
|       |— erc20.spec                   # Ghost-based ERC20 token model
|       |— helper.spec                  # CVL helper utilities
|       |— morpho_math_lib.spec          # MathLib CVL summaries
|       |— morpho_shares_math_lib.spec   # SharesMathLib CVL summaries
|       |— solmate_safe_transfer_lib.spec # SafeTransferLib CVL summaries
|   |— debug/                           # Debug/test specs
|       |— morpho_erc20_max_users_test.spec
|       |— morpho_sanity.spec
|       |— morpho_valid_state_debug.spec

```

Assumptions

Formal verification requires assumptions about the code and its environment to address prover timeouts, tool limitations, and state consistency. However, incorrect assumptions can mask real bugs by excluding reachable states from analysis. To maintain transparency, all assumptions are categorized into four groups: **Safe** (real-world constraints that don't reduce security coverage), **Proved** (formally verified invariants reused as preconditions), **Unsafe** (scope reductions necessary for tractability that may exclude valid scenarios), and **Trusted** (initialization state and admin-configured parameters assumed correct because initialization logic is excluded from verification).

Safe Assumptions

These reflect real-world constraints that don't impact security guarantees. In the codebase, every `require` statement that constitutes a safe assumption is annotated with a `"SAFE: ..."` message string.

Environment Constraints ([env.spec](#)):

Morpho has no payable functions, so `msg.value == 0` is assumed. Sender is non-zero and not the Morpho contract itself. Timestamps are bounded between realistic values (greater than 0, less than `max uint32`). Block number is non-zero. When two environments are used in the same rule, they share the same block context.

- `msg.value == 0` (no ETH)
- `msg.sender != 0` (non-zero sender)
- `msg.sender != currentContract` (sender is not Morpho)
- `block.timestamp` bounded between `max_uint16` and `max_uint32`
- `block.number != 0`
- Same-block constraints for multi-env rules

Ghost Synchronization ([morpho.spec](#)):

All ghost variables tracking Morpho's storage are synchronized with actual storage values via `require` statements in Sload hooks. These are safe because the ghost-hook mechanism ensures one-to-one correspondence between ghost state and contract storage.

- Ghost sync for all market fields (totalSupplyAssets, totalSupplyShares, totalBorrowAssets, totalBorrowShares, lastUpdate, fee)

- Ghost sync for position fields (supplyShares, borrowShares, collateral)
- Ghost sync for global state (owner, feeRecipient, isIrmEnabled, isLltvEnabled, isAuthorized, nonce)
- Ghost sync for market params (loanToken, collateralToken, oracle, irm, lltv)
- User account bounded set membership for position access

ERC20 Model ([erc20.spec](#)):

ERC20 tokens are modeled with standard assumptions: total supply equals sum of all balances, token decimals between 6 and 18, called contract is non-zero, accounts are within predefined bounded sets.

- Total supply equals sum of all balances
- Realistic token decimals (6-18)
- Non-zero token contract address
- Account set membership for transfers and approvals

Math Library Summaries ([morpho_math_lib.spec](#)):

Division denominators are assumed non-zero, matching Solidity's revert behavior for division by zero.

- `mulDivDown` denominator non-zero
- `mulDivUp` denominator non-zero

Proved Assumptions

These properties have been formally verified as valid state invariants and are used as trusted preconditions (via `requireInvariant`) in state transition and high-level rules. See [Valid State](#) for detailed descriptions and prover run links.

Accounting Consistency ([morpho_valid_state_many.spec](#)):

- Total supply assets are at least total borrow assets (`liquidityInvariant`)
- Sum of user supply shares does not exceed total supply shares (`supplySharesSolvency`)
- Sum of user borrow shares does not exceed total borrow shares (`borrowSharesSolvency`)
- Fee is bounded by MAX_FEE (25%) (`feeBounded`)

Market Existence ([morpho_valid_state_many.spec](#)):

- Non-existent markets have zero totals (`nonExistentMarketIsZero`)
- Non-existent markets have zero params (`nonExistentMarketParamsAreZero`)
- Non-existent markets have zero positions (`nonExistentMarketPositionsZero`)
- Supply/borrow shares and collateral require market existence

Market Configuration ([morpho_valid_state_many.spec](#)):

- Market IRM is enabled (`marketIrmIsEnabled`)
- Market LLTV is enabled (`marketLltvIsEnabled`)
- Enabled LLTVs are below WAD (`enabledLltvBelowWad`)
- Market LLTV is below WAD (`marketLltvBelowWad`)

Temporal Bounds ([morpho_valid_state_many.spec](#)):

- Last update is bounded by current timestamp (`lastUpdateBoundedByTimestamp`)
- Last update has a minimum bound (`lastUpdateMinBound`)

Safety (`morpho_valid_state_many.spec`):

- Borrowers always have collateral (`alwaysCollateralized`)
- Zero address does not authorize (`zeroDoesNotAuthorize`)

Unsafe Assumptions

These reduce verification scope to make the problem tractable for the prover. In the codebase, every `require` statement that constitutes an unsafe assumption is annotated with an `"UNSAFE: ..."` message string.

ERC20 Transfer Bounds (`erc20.spec`):

Transfer amounts in the ERC20 CVL model are bounded by max uint128 to avoid overflow in ghost arithmetic. This excludes scenarios with extremely large token amounts but is necessary for prover tractability.

- Transfer amount bounded by max uint128
- TransferFrom amount bounded by max uint128
- SafeTransfer `to` bounded to predefined account set
- SafeTransferFrom `from` bounded to predefined account set

Prover Configuration:

- Loop unrolling is capped at 3 iterations across all configurations
- Optimistic loop is enabled (assumes loop termination)

One-Market Regime Pins (`morpho_one.spec`):

The one-market regime collapses per-market storage to scalar (no-id) ghosts. Sload mirror checks on `_Morpho.market[KEY id].*`, `_Morpho.position[KEY id][KEY user].*`, and `_Morpho.idToMarketParams[KEY id].*` slots require the stored value to match the scalar ghost, making paths that touch a second distinct id with a conflicting value infeasible. The regime also pins `ghostMBOneLoanToken != 0` so `ERC20_ACCOUNT_BOUNDS` on position hooks stays bound to a stable loan token. These constraints exclude verification paths that genuinely cross between markets; cross-id reasoning is delegated to the many-market regime.

- Scalar `ghostMBOne*` ghosts mirror only one symbolic market.
- Sload mirror on per-market slots forces single-id consistency.
- `ghostMBOneLoanToken != 0` pre-state require in `setupOneBlue`.

Trusted Assumptions

These assume that governance and market creation have been done correctly with reasonable parameters. In the codebase, every `require` statement that constitutes a trusted assumption is annotated with a `"TRUSTED: ..."` message string.

Governance State (`morpho.spec` / `morpho_many.spec` / `morpho_one.spec`):

The protocol owner is assumed to be non-zero, reflecting that `setOwner(address(0))` would not be called in practice. The constructor enforces `owner != address(0)`, but `setOwner` only checks `newOwner != owner`, not `newOwner != 0`. Governance discipline prevents self-destructing ownership.

- Owner is non-zero

Market Initialization ([morpho_many.spec](#) / [morpho_one.spec](#)):

Created markets are assumed to have a non-zero loan token. `createMarket` does not validate `loanToken != address(0)`, but a market with zero-address loan token would be non-functional (`safeTransfer` reverts). Market creators are trusted not to use zero-address tokens. The constraint is expressed as `forall id.`

`lastUpdate[id] != 0 => loanToken[id] != 0` in the many-market regime and as `ghostMBOneLastUpdate128 != 0 => ghostMBOneLoanToken != 0` in the one-market regime.

- Created markets have non-zero loan token

Verification Properties

Links to specific CVL spec files are provided for each property, with status indicators.

-  Verified
-  Timeout
-  Violated

High-Level

Complex multi-step properties verifying Morpho Blue's business logic integrity. These target specific function sequences to validate share conservation, asset conservation, round-trip safety, and protocol safety invariants.

Property	Name	Description	Status	Notes
HL-01	<code>supplySharesConservation</code>	Supply: total shares delta == user delta + fee recipient delta <code>supply(): totalSharesDelta == userSharesDelta + feeRecSharesDelta</code>		
HL-02	<code>withdrawSharesConservation</code>	Withdraw: total shares delta == user delta + fee recipient delta <code>withdraw(): totalSharesDelta == userSharesDelta + feeRecSharesDelta</code>		
HL-03	<code>borrowSharesConservation</code>	Borrow: total borrow shares delta == user borrow shares delta <code>borrow(): totalBorrowSharesDelta == userBorrowSharesDelta</code>		Mutation tested (7/10)
HL-04	<code>repaySharesConservation</code>	Repay: total borrow shares delta == user borrow shares delta <code>repay(): totalBorrowSharesDelta == userBorrowSharesDelta</code>		
HL-05	<code>supplyAssetConservation</code>	Supply: net asset delta (minus interest) == ERC20 balance change <code>supply(): (supplyAssetsDelta - interestDelta) == erc20BalanceDelta</code>		
HL-06	<code>supplyWithdrawNoProfit</code>	Supply-then-withdraw round-trip yields no profit <code>supply(assets); withdraw(shares) => balanceAfter <= balanceBefore</code>		
HL-07	<code>borrowRepayNoProfit</code>	Borrow-then-repay round-trip costs at least what was borrowed <code>borrow(assets); repay(shares) => balanceAfter <= balanceBefore</code>		
HL-08	<code>accrueInterestIncreasesSupplyAssets</code>	Interest accrual monotonically increases supply assets <code>accrueInterest() => totalSupplyAssetsAfter >= totalSupplyAssetsBefore</code>		

Property	Name	Description	Status	Notes
HL-09	<code>accrueInterestIncreasesBorrowAssets</code>	Interest accrual monotonically increases borrow assets <code>accrueInterest() =></code> <code>totalBorrowAssetsAfter >=</code> <code>totalBorrowAssetsBefore</code>	✓	
HL-10	<code>accrueInterestEqualDelta</code>	Interest accrual increases supply and borrow assets by equal amounts <code>accrueInterest() =></code> <code>supplyAssetsDelta == borrowAssetsDelta</code>	✓	
HL-11	<code>liquidationPreservesLiquidityRelation</code>	Liquidation preserves <code>totalBorrowAssets <= totalSupplyAssets</code> <code>liquidate() =></code> <code>totalBorrowAssetsAfter <= totalSupplyAssetsAfter</code>	✓	
HL-12	<code>liquidationBorrowSharesConservation</code>	Liquidation: total borrow shares delta == borrower shares delta <code>liquidate() =></code> <code>totalBorrowSharesDelta == borrowerSharesDelta</code>	✓	
HL-13	<code>supplyCollateralDoesNotChangeTotals</code>	Supply collateral does not change market-level supply or borrow totals <code>supplyCollateral() =></code> all totals unchanged	✓	
HL-14	<code>nonLiquidationPreservesCollateralization</code>	Non-liquidation operations preserve position collateralization <code>forall f: f != liquidate AND</code> <code>borrowShares > 0 => collateral > 0</code>	✓	

Valid State

System-wide invariants that define the fundamental constraints of Morpho Blue's accounting and structural integrity. These 23 invariants are verified parametrically against every external function and serve as trusted preconditions for other property categories.

All 23 invariants are verified in both the one-market regime (`morpho_valid_state_one.spec` → `valid_state_one.conf`) and the many-market regime (`morpho_valid_state_many.spec` → `valid_state_many.conf`). The spec links in the table below point to the many-market form (with `forall id`) for readability; the one-market form is the same property with `forall id` dropped against scalar ghosts. See [Verification Approach](#).

Accounting Invariants

Property	Name	Description	Status	Notes
VS-01	<code>feeBounded</code>	Market fee never exceeds MAX_FEE (25%) <code>forall id: fee[id] <= MAX_FEE</code>	✓	
VS-02	<code>feeRequiresMarket</code>	Fee is zero for non-existent markets <code>forall id: fee[id] != 0 =></code> <code>lastUpdate[id] != 0</code>	✓	

Property	Name	Description	Status	Notes
VS-05	liquidityInvariant	Total supply assets >= total borrow assets for existing markets forall id: lastUpdate[id] != 0 => totalBorrowAssets[id] <= totalSupplyAssets[id]	✓	Mutation tested (4/13)
VS-06	supplySharesSolvency	Sum of user supply shares <= total supply shares forall id: totalSupplyShares[id] >= sum(supplyShares[id])	✓	Mutation tested (3/10)
VS-07	borrowSharesSolvency	Sum of user borrow shares <= total borrow shares forall id: totalBorrowShares[id] >= sum(borrowShares[id])	✓	

Temporal Invariants

Property	Name	Description	Status	Notes
VS-03	lastUpdateBoundedByTimestamp	Last update <= current block timestamp for existing markets forall id: lastUpdate[id] != 0 => lastUpdate[id] <= block.timestamp	✓	
VS-04	lastUpdateMinBound	Last update >= MIN_BLOCK_TIMESTAMP for existing markets forall id: lastUpdate[id] != 0 => lastUpdate[id] >= MIN_TIMESTAMP	✓	

Market Existence Invariants

Property	Name	Description	Status	Notes
VS-08	nonExistentMarketIsZero	Non-existent markets have zero accounting totals forall id: lastUpdate[id] == 0 => all totals and fee == 0	✓	
VS-09	nonExistentMarketParamsAreZero	Non-existent markets have zero market params forall id: lastUpdate[id] == 0 => all params == 0	✓	

Property	Name	Description	Status	Notes
VS-21	<code>nonExistentMarketPositionsZero</code>	Non-existent markets have zero positions <code>forall id, user:</code> <code>lastUpdate[id] == 0 =></code> <code>supplyShares == 0 AND</code> <code>borrowShares == 0 AND</code> <code>collateral == 0</code>	✓	

Market Configuration Invariants

Property	Name	Description	Status	Notes
VS-10	<code>marketIrmIsEnabled</code>	Created market's IRM is in the enabled set <code>forall id: lastUpdate[id] != 0 =></code> <code>isIrmEnabled[irm[id]]</code>	✓	
VS-11	<code>marketLltvIsEnabled</code>	Created market's LLTV is in the enabled set <code>forall id: lastUpdate[id] != 0 =></code> <code>isLltvEnabled[lltv[id]]</code>	✓	
VS-12	<code>enabledLltvBelowWad</code>	Enabled LLTVs are strictly below WAD <code>forall lltv: isLltvEnabled[lltv] =></code> <code>lltv < WAD</code>	✓	
VS-13	<code>marketLltvBelowWad</code>	Created market's LLTV is strictly below WAD <code>forall id: lastUpdate[id] != 0 =></code> <code>lltv[id] < WAD</code>	✓	

Position Existence Invariants

Property	Name	Description	Status	Notes
VS-14	<code>supplySharesRequiresMarket</code>	Non-zero supply shares imply market exists <code>forall id, user: supplyShares[id]</code> <code>[user] > 0 => lastUpdate[id] != 0</code>	✓	
VS-15	<code>borrowSharesRequiresMarket</code>	Non-zero borrow shares imply market exists <code>forall id, user: borrowShares[id]</code> <code>[user] > 0 => lastUpdate[id] != 0</code>	✓	
VS-16	<code>collateralRequiresMarket</code>	Non-zero collateral implies market exists <code>forall id, user: collateral[id][user]</code> <code>> 0 => lastUpdate[id] != 0</code>	✓	
VS-17	<code>supplyAssetsRequiresMarket</code>	Non-zero total supply assets imply market exists <code>forall id: totalSupplyAssets[id] > 0</code> <code>=> lastUpdate[id] != 0</code>	✓	

Property	Name	Description	Status	Notes
VS-18	<code>supplySharesTotalRequiresMarket</code>	Non-zero total supply shares imply market exists <code>forall id: totalSupplyShares[id] > 0 => lastUpdate[id] != 0</code>	✓	
VS-19	<code>borrowAssetsRequiresMarket</code>	Non-zero total borrow assets imply market exists <code>forall id: totalBorrowAssets[id] > 0 => lastUpdate[id] != 0</code>	✓	
VS-20	<code>borrowSharesTotalRequiresMarket</code>	Non-zero total borrow shares imply market exists <code>forall id: totalBorrowShares[id] > 0 => lastUpdate[id] != 0</code>	✓	

Safety Invariants

Property	Name	Description	Status	Notes
VS-22	<code>alwaysCollateralized</code>	Positions with borrow shares always have collateral <code>forall id, user: borrowShares[id][user] != 0 => collateral[id][user] != 0</code>	✓	
VS-23	<code>zeroDoesNotAuthorize</code>	Zero address does not authorize anyone <code>forall authorized: !isAuthorized[0][authorized]</code>	✓	

Variable Transitions

Properties verifying that specific storage variables in Morpho Blue change only under expected conditions. Each rule captures a variable value before and after an arbitrary function call, asserting it changed only as permitted.

Property	Name	Description	Status	Notes
VT-01	<code>irmEnablementIsPermanent</code>	Once an IRM is enabled, it stays enabled forever <code>forall f: isIrmEnabled[irm] before => isIrmEnabled[irm] after</code>	✓	
VT-02	<code>lltvEnablementIsPermanent</code>	Once an LLTV is enabled, it stays enabled forever <code>forall f: isLltvEnabled[lltv] before => isLltvEnabled[lltv] after</code>	✓	
VT-03	<code>marketParamsAddressesImmutableAfterCreation</code>	Market param addresses cannot change after creation <code>forall f, id: loanToken[id] != 0 => loanToken[id] unchanged (and collateralToken, oracle, irm)</code>	✓	

Property	Name	Description	Status	Notes
VT-04	marketLltvImmutableAfterCreation	Market LLTV cannot change after creation forall f, id: lltv[id] != 0 => lltv[id] unchanged	✓	
VT-05	lastUpdateOnlyIncreases	Last update timestamp only increases or stays the same forall f, id: lastUpdate[id] after >= lastUpdate[id] before	✓	
VT-06	lastUpdateNonZeroLatch	Once a market is created (lastUpdate non-zero), it cannot be uncreated forall f, id: lastUpdate[id] != 0 before => lastUpdate[id] != 0 after	✓	

State Transitions

Properties verifying the correctness of transitions between valid states. These ensure Morpho Blue's state machine operates correctly and that accounting changes are coherent across related variables.

Market Lifecycle

Property	Name	Description	Status	Notes
ST-01	marketCreationAtomicity	Market params only change during market creation (lastUpdate 0->non-zero) forall f: paramsChanged => (lastUpdate: 0 -> non-zero)	✓	
ST-02	accountingChangesRefreshTimestamp	Accounting changes with elapsed time refresh lastUpdate to block.timestamp forall f: accountingChanged AND timeElapsed => lastUpdate == block.timestamp	✓	
ST-03	lastUpdateChangeRequiresAccountingOrFeeChange	lastUpdate change requires accounting change, fee change, or timestamp refresh forall f: lastUpdateChanged => created OR accountingChanged OR feeChanged OR refreshed	✓	

Supply/Borrow Share Co-transitions

Property	Name	Description	Status	Notes
ST-04	userSupplySharesIncreaseImpliesTotalIncrease	User supply shares increase implies total supply shares increase forall f: supplyShares[id][user] increased => totalSupplyShares[id] increased	✓	
ST-05	userSupplySharesDecreaseImpliesTotalDecrease	User supply shares decrease implies total supply shares decrease forall f: supplyShares[id][user] decreased => totalSupplyShares[id] decreased	✓	

Property	Name	Description	Status	Notes
ST-06	<code>userBorrowSharesIncreaseImpliesTotalIncrease</code>	User borrow shares increase implies total borrow shares increase <code>forall f: borrowShares[id][user] increased => totalBorrowShares[id] increased</code>	✓	
ST-07	<code>userBorrowSharesDecreaseImpliesTotalDecrease</code>	User borrow shares decrease implies total borrow shares decrease <code>forall f: borrowShares[id][user] decreased => totalBorrowShares[id] decreased</code>	✓	

Asset/Share Correlation

Property	Name	Description	Status	Notes
ST-08	<code>totalSupplySharesIncreaseImpliesAssetsIncrease</code>	Supply shares increase implies supply assets increase <code>forall f: totalSupplyShares[id] increased => totalSupplyAssets[id] increased</code>	✓	
ST-09	<code>totalSupplySharesDecreaseImpliesAssetsDecrease</code>	Supply shares decrease implies supply assets do not increase <code>forall f: totalSupplyShares[id] decreased => totalSupplyAssets[id] <= before</code>	✓	
ST-10	<code>totalBorrowSharesIncreaseImpliesAssetsIncrease</code>	Borrow shares increase implies borrow assets do not decrease <code>forall f: totalBorrowShares[id] increased => totalBorrowAssets[id] >= before</code>	✓	

Cross-variable Consistency

Property	Name	Description	Status	Notes
ST-11	<code>interestAccrualSymmetry</code>	Pure interest accrual increases supply assets by at least the borrow assets delta <code>accrueInterest(): pureInterest => supplyAssetsDelta >= borrowAssetsDelta</code>	✓	
ST-12	<code>collateralAndBorrowDecreaseImpliesTotalBorrowSharesDecrease</code>	Collateral and borrow shares decrease imply total borrow shares decrease <code>forall f: collateral decreased AND borrowShares decreased => totalBorrowShares decreased</code>	✓	
ST-13	<code>supplyIncreasesContractBalance</code>	Supply shares increase is accompanied by loan token balance increase <code>supply(): totalSupplyShares increased => loanToken.balanceOf(Morpho) increased</code>	✓	

Unit Tests

Properties verifying basic behavior of individual Morpho Blue functions: direct effects on state, non-effects on unrelated state, view function consistency, and never-revert guarantees.

Direct Effects

Property	Name	Description	Status	Notes
UT-01	<code>setOwnerSetsNewOwner</code>	setOwner sets the new owner <code>setOwner(newOwner) => owner == newOwner</code>	✓	
UT-02	<code>enableIrmEnablesGivenIrm</code>	enableIrm enables the given IRM <code>enableIrm(irm) => isIrmEnabled[irm] == true</code>	✓	
UT-03	<code>enableLltvEnablesGivenLltv</code>	enableLltv enables the given LLTV <code>enableLltv(lltv) => isLltvEnabled[lltv] == true</code>	✓	
UT-04	<code>setFeeRecipientSetsNewRecipient</code>	setFeeRecipient sets the new recipient <code>setFeeRecipient(addr) => feeRecipient == addr</code>	✓	
UT-05	<code>setFeeSetsNewFee</code>	setFee sets the new fee for the market <code>setFee(mp, newFee) => fee[id] == newFee</code>	✓	
UT-06	<code>createMarketSetsLastUpdate</code>	createMarket sets lastUpdate to block.timestamp <code>createMarket(mp) => lastUpdate[id] == block.timestamp</code>	✓	
UT-07	<code>createMarketStoresParams</code>	createMarket stores all market params <code>createMarket(mp) => idToMarketParams[id] matches mp</code>	✓	
UT-08	<code>supplyIncreasesSharesAndTotals</code>	Supply increases user shares and market totals <code>supply() => supplyShares, totalSupplyShares, totalSupplyAssets all >= before</code>	✓	
UT-09	<code>withdrawDecreasesSharesAndTotals</code>	Withdraw decreases user shares by returned amount <code>withdraw() => supplySharesAfter == supplySharesBefore - returnedShares</code>	✓	
UT-10	<code>borrowIncreasesBorrowSharesAndTotals</code>	Borrow increases user borrow shares by returned amount <code>borrow() => borrowSharesAfter == borrowSharesBefore + returnedShares</code>	✓	
UT-11	<code>repayDecreasesBorrowShares</code>	Repay decreases user borrow shares by returned amount <code>repay() => borrowSharesAfter == borrowSharesBefore - returnedShares</code>	✓	
UT-12	<code>supplyCollateralIncreasesUserCollateral</code>	Supply collateral increases user collateral by assets <code>supplyCollateral/assets) => collateralAfter == collateralBefore + assets</code>	✓	
UT-13	<code>withdrawCollateralDecreasesUserCollateral</code>	Withdraw collateral decreases user collateral by assets <code>withdrawCollateral/assets) => collateralAfter == collateralBefore - assets</code>	✓	
UT-14	<code>setAuthorizationSetsFlag</code>	setAuthorization sets the authorization flag <code>setAuthorization(authorized, val) => isAuthorized[sender][authorized] == val</code>	✓	
UT-15	<code>liquidateDecreasesBorrowerCollateral</code>	Liquidate does not increase borrower collateral <code>liquidate() => collateralAfter <= collateralBefore</code>	✓	
UT-16	<code>liquidateDecreasesBorrowerBorrowShares</code>	Liquidate does not increase borrower borrow shares <code>liquidate() => borrowSharesAfter <= borrowSharesBefore</code>	✓	

Non-Effects

Property	Name	Description	Status	Notes
UT-17	<code>setOwnerDoesNotChangeFeeRecipient</code>	setOwner does not change fee recipient <code>setOwner() => feeRecipient unchanged</code>	✓	
UT-18	<code>setFeeRecipientDoesNotChangeOwner</code>	setFeeRecipient does not change owner <code>setFeeRecipient() => owner unchanged</code>	✓	
UT-19	<code>enableIrmDoesNotAffectLltvEnablement</code>	enableIrm does not affect LLTV enablement <code>enableIrm(irm) => isLltvEnabled[lttv] unchanged</code>	✓	
UT-20	<code>enableLltvDoesNotAffectIrmEnablement</code>	enableLltv does not affect IRM enablement <code>enableLltv(ltv) => isIrmEnabled[irm] unchanged</code>	✓	
UT-21	<code>supplyCollateralDoesNotChangeSupplyShares</code>	Supply collateral does not change any user's supply shares <code>supplyCollateral() => supplyShares[id][user] unchanged</code>	✓	
UT-22	<code>supplyCollateralDoesNotChangeBorrowTotals</code>	Supply collateral does not change borrow totals <code>supplyCollateral() => totalBorrowAssets[id], totalBorrowShares[id] unchanged</code>	✓	
UT-23	<code>supplyCollateralDoesNotChangeSupplyTotals</code>	Supply collateral does not change supply totals <code>supplyCollateral() => totalSupplyAssets[id], totalSupplyShares[id] unchanged</code>	✓	
UT-24	<code>setAuthorizationDoesNotAffectOtherPairs</code>	setAuthorization does not affect other authorization pairs <code>setAuthorization(authorized, val) => isAuthorized[other1][other2] unchanged</code>	✓	

Third-Party Isolation

Property	Name	Description	Status	Notes
UT-25	<code>supplyDoesNotAffectThirdPartyShares</code>	Supply does not affect third-party supply shares <code>supply(onBehalf) => supplyShares[id][third] unchanged (third != onBehalf AND third != feeRecipient)</code>	✓	
UT-26	<code>withdrawDoesNotAffectThirdPartyShares</code>	Withdraw does not affect third-party supply shares <code>withdraw(onBehalf) => supplyShares[id][third] unchanged (third != onBehalf AND third != feeRecipient)</code>	✓	
UT-27	<code>borrowDoesNotAffectThirdPartyBorrowShares</code>	Borrow does not affect third-party borrow shares <code>borrow(onBehalf) => borrowShares[id][third] unchanged (third != onBehalf)</code>	✓	
UT-28	<code>repayDoesNotAffectThirdPartyBorrowShares</code>	Repay does not affect third-party borrow shares <code>repay(onBehalf) => borrowShares[id][third] unchanged (third != onBehalf)</code>	✓	
UT-29	<code>supplyCollateralDoesNotAffectThirdPartyCollateral</code>	Supply collateral does not affect third-party collateral <code>supplyCollateral(onBehalf) => collateral[id][third] unchanged (third != onBehalf)</code>	✓	
UT-30	<code>withdrawCollateralDoesNotAffectThirdPartyCollateral</code>	Withdraw collateral does not affect third-party collateral <code>withdrawCollateral(onBehalf) => collateral[id][third] unchanged (third != onBehalf)</code>	✓	

Cross-Market Isolation

Property	Name	Description	Status	Notes
UT-31	<code>setFeeDoesNotCorruptOtherMarketLastUpdate</code>	setFee does not corrupt other market's lastUpdate (packed slot) <code>setFee(mp) => lastUpdate[otherId]</code> unchanged	✓	
UT-32	<code>setFeeDoesNotCorruptOtherMarketTotals</code>	setFee does not corrupt other market's totals <code>setFee(mp) => all totals[otherId]</code> unchanged	✓	
UT-33	<code>supplyDoesNotAffectOtherMarketTotals</code>	Supply does not affect other market's totals <code>supply(mp) => all totals[otherId]</code> unchanged	✓	
UT-34	<code>borrowDoesNotAffectOtherMarketTotals</code>	Borrow does not affect other market's totals <code>borrow(mp) => all totals[otherId]</code> unchanged	✓	

View Function Consistency

Property	Name	Description	Status	Notes
UT-35	<code>ownerSameForAnyCaller</code>	owner() returns the same value for any caller <code>owner(e1) == owner(e2)</code> for any e1, e2	✓	
UT-36	<code>feeRecipientSameForAnyCaller</code>	feeRecipient() returns the same value for any caller <code>feeRecipient(e1) == feeRecipient(e2)</code> for any e1, e2	✓	
UT-37	<code>isIrmEnabledSameForAnyCaller</code>	isIrmEnabled() returns the same value for any caller <code>isIrmEnabled(irm, e1) == isIrmEnabled(irm, e2)</code> for any e1, e2	✓	
UT-38	<code>isLltvEnabledSameForAnyCaller</code>	isLltvEnabled() returns the same value for any caller <code>isLltvEnabled(lltv, e1) == isLltvEnabled(lltv, e2)</code> for any e1, e2	✓	

Never-Revert Guarantees

Property	Name	Description	Status	Notes
UT-39	<code>ownerNeverReverts</code>	owner() never reverts <code>@withrevert owner(); !lastReverted</code>	✓	

Property	Name	Description	Status	Notes
UT-40	<code>feeRecipientNeverReverts</code>	feeRecipient() never reverts @withrevert feeRecipient(); !lastReverted	✓	
UT-41	<code>isIrmEnabledNeverReverts</code>	isIrmEnabled() never reverts @withrevert isIrmEnabled(irm); !lastReverted	✓	
UT-42	<code>isLltvEnabledNeverReverts</code>	isLltvEnabled() never reverts @withrevert isLltvEnabled(lltv); !lastReverted	✓	
UT-43	<code>nonceNeverReverts</code>	nonce() never reverts @withrevert nonce(addr); !lastReverted	✓	

Accrue Interest

Properties verifying that interest accrual is correctly triggered across lending operations. Each rule uses `lastStorage` to compare two execution paths: one with an explicit `accrueInterest()` call before the operation, and one without. Identity of final storage proves the operation internally calls `_accrueInterest()`.

Property	Name	Description	Status	Notes
AI-01	<code>supplyAccruesInterest</code>	Explicit accrueInterest before supply is idempotent <code>storage(accrueInterest + supply) == storage(supply alone)</code>	✓	
AI-02	<code>withdrawAccruesInterest</code>	Explicit accrueInterest before withdraw is idempotent <code>storage(accrueInterest + withdraw) == storage(withdraw alone)</code>	✓	
AI-03	<code>borrowAccruesInterest</code>	Explicit accrueInterest before borrow is idempotent <code>storage(accrueInterest + borrow) == storage(borrow alone)</code>	✓	
AI-04	<code>repayAccruesInterest</code>	Explicit accrueInterest before repay is idempotent <code>storage(accrueInterest + repay) == storage(repay alone)</code>	✓	

Reachability

Properties using `satisfy()` to prove that key scenarios are reachable, confirming non-vacuous verification. Basic function reachability confirms each function can execute successfully; conditional reachability validates that specific protocol states and edge cases are achievable.

Basic Function Reachability

Property	Name	Description	Status	Notes
RC-01	<code>setOwnerIsReachable</code>	setOwner can execute successfully <code>satisfy(!lastReverted)</code>	✓	
RC-02	<code>enableIrmIsReachable</code>	enableIrm can execute successfully <code>satisfy(!lastReverted)</code>	✓	
RC-03	<code>enableLltvIsReachable</code>	enableLltv can execute successfully <code>satisfy(!lastReverted)</code>	✓	
RC-04	<code>setFeeIsReachable</code>	setFee can execute successfully <code>satisfy(!lastReverted)</code>	✓	
RC-05	<code>setFeeRecipientIsReachable</code>	setFeeRecipient can execute successfully <code>satisfy(!lastReverted)</code>	✓	
RC-06	<code>createMarketIsReachable</code>	createMarket can execute successfully <code>satisfy(!lastReverted)</code>	✓	
RC-07	<code>supplyIsReachable</code>	supply can execute successfully <code>satisfy(!lastReverted)</code>	✓	
RC-08	<code>withdrawIsReachable</code>	withdraw can execute successfully <code>satisfy(!lastReverted)</code>	✓	
RC-09	<code>borrowIsReachable</code>	borrow can execute successfully <code>satisfy(!lastReverted)</code>	✓	
RC-10	<code>repayIsReachable</code>	repay can execute successfully <code>satisfy(!lastReverted)</code>	✓	
RC-11	<code>supplyCollateralIsReachable</code>	supplyCollateral can execute successfully <code>satisfy(!lastReverted)</code>	✓	
RC-12	<code>withdrawCollateralIsReachable</code>	withdrawCollateral can execute successfully <code>satisfy(!lastReverted)</code>	✓	
RC-13	<code>liquidateIsReachable</code>	liquidate can execute successfully <code>satisfy(!lastReverted)</code>	✓	
RC-14	<code>flashLoanIsReachable</code>	flashLoan can execute successfully <code>satisfy(!lastReverted)</code>	✓	
RC-15	<code>setAuthorizationIsReachable</code>	setAuthorization can execute successfully <code>satisfy(!lastReverted)</code>	✓	
RC-16	<code>accrueInterestIsReachable</code>	accrueInterest can execute successfully <code>satisfy(!lastReverted)</code>	✓	

Conditional Reachability

Property	Name	Description	Status	Notes
RC-17	<code>supplyReachableWithPositiveAssets</code>	Supply with positive assets is reachable <code>satisfy(totalSupplyAssetsAfter > totalSupplyAssetsBefore)</code>	✓	
RC-18	<code>borrowReachableWithCollateral</code>	Borrow with existing collateral is reachable <code>satisfy(totalBorrowAssetsAfter > totalBorrowAssetsBefore)</code>	✓	
RC-19	<code>liquidateReachableWithBadDebt</code>	Liquidation with bad debt socialization is reachable <code>satisfy(totalSupplyAssetsAfter < totalSupplyAssetsBefore)</code>	✓	
RC-20	<code>withdrawReachableByAuthorizedAgent</code>	Withdraw by authorized agent is reachable <code>satisfy(sender != onBehalf AND isAuthorized[onBehalf][sender] AND sharesDecreased)</code>	✓	
RC-21	<code>setFeeReachableWithZeroFee</code>	Fee removal (set to zero) is reachable <code>satisfy(fee[id] > 0 before AND fee[id] == 0 after)</code>	✓	
RC-22	<code>accrueInterestReachableWithPositiveInterest</code>	Interest accrual with positive interest is reachable <code>satisfy(totalBorrowAssetsAfter > totalBorrowAssetsBefore)</code>	✓	
RC-23	<code>repayReachableWithFullRepayment</code>	Full debt repayment is reachable <code>satisfy(borrowShares[id][onBehalf] > 0 before AND borrowShares[id][onBehalf] == 0 after)</code>	✓	
RC-24	<code>canWithdrawAll</code>	Full supply withdrawal with active borrows is reachable <code>satisfy(borrows > 0 AND otherSuppliers exist AND supplyShares[id][user] == 0 after)</code>	✓	
RC-25	<code>canWithdrawCollateralAll</code>	Full collateral withdrawal (debt-free) is reachable <code>satisfy(collateral[id][user] > 0 before AND collateral[id][user] == 0 after)</code>	✓	

Reverts

Properties verifying revert conditions using `@withrevert` and `lastReverted`. These confirm that Morpho Blue functions correctly reject invalid inputs, enforce preconditions, and revert under expected circumstances.

Governance Reverts

Property	Name	Description	Status	Notes
RV-02	<code>setOwnerRevertsForNonOwner</code>	setOwner reverts when caller is not the owner <code>@withrevert setOwner(); msg.sender != owner => lastReverted</code>	✓	
RV-03	<code>setOwnerRevertsWhenAlreadySet</code>	setOwner reverts when newOwner equals current owner <code>@withrevert setOwner(newOwner); newOwner == owner => lastReverted</code>	✓	
RV-04	<code>enableIrmRevertsForNonOwner</code>	enableIrm reverts when caller is not the owner <code>@withrevert enableIrm(); msg.sender != owner => lastReverted</code>	✓	
RV-05	<code>enableIrmRevertsWhenAlreadyEnabled</code>	enableIrm reverts when IRM is already enabled <code>@withrevert enableIrm(irm); isIrmEnabled[irm] => lastReverted</code>	✓	

Property	Name	Description	Status	Notes
RV-06	<code>enableLltvRevertsForNonOwner</code>	enableLltv reverts when caller is not the owner <code>@withrevert enableLltv(); msg.sender != owner => lastReverted</code>	✓	
RV-07	<code>enableLltvRevertsWhenAlreadyEnabled</code>	enableLltv reverts when LLTV is already enabled <code>@withrevert enableLltv(lltv); isLltvEnabled[lltv] => lastReverted</code>	✓	
RV-08	<code>enableLltvRevertsWhenExceedsMax</code>	enableLltv reverts when LLTV >= WAD <code>@withrevert enableLltv(lltv); lltv >= WAD => lastReverted</code>	✓	
RV-09	<code>setFeeRevertsForNonOwner</code>	setFee reverts when caller is not the owner <code>@withrevert setFee(); msg.sender != owner => lastReverted</code>	✓	
RV-10	<code>setFeeRevertsWhenExceedsMax</code>	setFee reverts when fee exceeds MAX_FEE <code>@withrevert setFee(mp, newFee); newFee > MAX_FEE => lastReverted</code>	✓	
RV-11	<code>setFeeRevertsOnNonExistentMarket</code>	setFee reverts when market does not exist <code>@withrevert setFee(mp); lastUpdate[id] == 0 => lastReverted</code>	✓	
RV-12	<code>setFeeRecipientRevertsForNonOwner</code>	setFeeRecipient reverts when caller is not the owner <code>@withrevert setFeeRecipient(); msg.sender != owner => lastReverted</code>	✓	
RV-13	<code>setFeeRecipientRevertsWhenAlreadySet</code>	setFeeRecipient reverts when recipient is already set <code>@withrevert setFeeRecipient(addr); addr == feeRecipient => lastReverted</code>	✓	

Market Creation Reverts

Property	Name	Description	Status	Notes
RV-14	<code>createMarketRevertsWhenIrmNotEnabled</code>	createMarket reverts when IRM is not enabled <code>@withrevert createMarket(mp); !isIrmEnabled[mp.irm] => lastReverted</code>	✓	
RV-15	<code>createMarketRevertsWhenLltvNotEnabled</code>	createMarket reverts when LLTV is not enabled <code>@withrevert createMarket(mp); !isLltvEnabled[mp.lltv] => lastReverted</code>	✓	
RV-16	<code>createMarketRevertsWhenAlreadyCreated</code>	createMarket reverts when market already exists <code>@withrevert createMarket(mp); lastUpdate[id] != 0 => lastReverted</code>	✓	

Lending Operation Reverts

Property	Name	Description	Status	Notes
RV-17	<code>supplyRevertsOnNonExistentMarket</code>	supply reverts on non-existent market <code>@withrevert supply(mp); lastUpdate[id] == 0 => lastReverted</code>	✓	

Property	Name	Description	Status	Notes
RV-18	<code>supplyRevertsForZeroOnBehalf</code>	supply reverts when onBehalf is zero address <code>@withrevert supply(onBehalf=0);</code> <code>lastReverted</code>	✓	
RV-19	<code>withdrawRevertsForZeroReceiver</code>	withdraw reverts when receiver is zero address <code>@withrevert withdraw(receiver=0);</code> <code>lastReverted</code>	✓	
RV-20	<code>withdrawRevertsWhenUnauthorized</code>	withdraw reverts when sender is unauthorized <code>@withrevert withdraw(); sender !=</code> <code>onBehalf AND !isAuthorized =></code> <code>lastReverted</code>	✓	
RV-21	<code>borrowRevertsForZeroReceiver</code>	borrow reverts when receiver is zero address <code>@withrevert borrow(receiver=0);</code> <code>lastReverted</code>	✓	
RV-22	<code>borrowRevertsWhenUnauthorized</code>	borrow reverts when sender is unauthorized <code>@withrevert borrow(); sender !=</code> <code>onBehalf AND !isAuthorized =></code> <code>lastReverted</code>	✓	
RV-23	<code>repayRevertsForZeroOnBehalf</code>	repay reverts when onBehalf is zero address <code>@withrevert repay(onBehalf=0);</code> <code>lastReverted</code>	✓	

Collateral Operation Reverts

Property	Name	Description	Status	Notes
RV-24	<code>supplyCollateralRevertsForZeroAssets</code>	supplyCollateral reverts when assets is zero <code>@withrevert supplyCollateral(assets=0); lastReverted</code>	✓	
RV-25	<code>supplyCollateralRevertsForZeroOnBehalf</code>	supplyCollateral reverts when onBehalf is zero address <code>@withrevert supplyCollateral(onBehalf=0);</code> <code>lastReverted</code>	✓	
RV-26	<code>withdrawCollateralRevertsForZeroAssets</code>	withdrawCollateral reverts when assets is zero <code>@withrevert withdrawCollateral(assets=0);</code> <code>lastReverted</code>	✓	
RV-27	<code>withdrawCollateralRevertsForZeroReceiver</code>	withdrawCollateral reverts when receiver is zero address <code>@withrevert withdrawCollateral(receiver=0);</code> <code>lastReverted</code>	✓	
RV-28	<code>withdrawCollateralRevertsWhenUnauthorized</code>	withdrawCollateral reverts when sender is unauthorized <code>@withrevert withdrawCollateral(); sender != onBehalf</code> <code>AND !isAuthorized => lastReverted</code>	✓	

Miscellaneous Reverts

Property	Name	Description	Status	Notes
RV-01	<code>isAuthorizedNeverReverts</code>	isAuthorized() never reverts <code>@withrevert isAuthorized(a, b);</code> <code>!lastReverted</code>	✓	
RV-29	<code>setAuthorizationRevertsWhenAlreadySet</code>	setAuthorization reverts when value is already set <code>@withrevert setAuthorization(addr, val);</code> <code>isAuthorized[sender][addr] == val =></code> <code>lastReverted</code>	✓	
RV-30	<code>flashLoanRevertsForZeroAssets</code>	flashLoan reverts when assets is zero <code>@withrevert flashLoan(token, assets=0,</code> <code>data); lastReverted</code>	✓	
RV-31	<code>accrueInterestRevertsOnNonExistentMarket</code>	accrueInterest reverts when market does not exist <code>@withrevert accrueInterest(mp);</code> <code>lastUpdate[id] == 0 => lastReverted</code>	✓	

Access Control

Parametric rules verifying that unauthorized callers cannot modify protected state in Morpho Blue. These confirm that position changes require proper authorization.

Property	Name	Description	Status	Notes
AC-01	<code>unauthorizedCannotDecreaseSupplyShares</code>	Unauthorized callers cannot decrease a user's supply shares <code>forall f: sender != user AND</code> <code>!isAuthorized[user][sender] =></code> <code>supplyShares[id][user] >= before</code>	✓	
AC-02	<code>unauthorizedCannotIncreaseBorrowShares</code>	Unauthorized callers cannot increase a user's borrow shares <code>forall f: sender != user AND</code> <code>!isAuthorized[user][sender] =></code> <code>borrowShares[id][user] <= before</code>	✓	
AC-03	<code>unauthorizedCannotDecreaseCollateral</code>	Unauthorized non-liquidation callers cannot decrease a user's collateral <code>forall f != liquidate: sender != user</code> <code>AND !isAuthorized => collateral[id]</code> <code>[user] >= before</code>	✓	

Mutation Testing

What is Mutation Testing

Mutation testing validates the strength and completeness of formal verification specifications by introducing small, deliberate bugs (mutations) into the source code and checking whether existing properties detect them. A **caught** mutation means the specification correctly identified the bug (the rule was violated). A **survived** mutation means the specification did not detect the change, indicating a potential gap in coverage.

Three properties were selected for mutation testing based on their criticality to Morpho Blue's accounting integrity.

liquidityInvariant

The `liquidityInvariant` (invariant in `morpho_valid_state_many.spec`) asserts that total borrow assets never exceed total supply assets. This is the core solvency constraint -- if violated, the protocol cannot honor all withdrawals.

Result: [4/13 mutations caught](#) -- the invariant detects sign flips and missing accounting on the supply side, but single-sided removals on borrow or withdraw survive because the inequality remains satisfiable.

Configuration: `valid_state_mutations_liquidityInvariant.conf`

supply mutations

#1  Removed totalSupplyAssets increase in supply.

```
-      market[id].totalSupplyAssets += assets.toUint128();
+      // market[id].totalSupplyAssets += assets.toUint128(); // MUTATION: removed
totalSupplyAssets increase in supply
```

#2  Flipped += to -= for totalSupplyAssets in supply.

```
-      market[id].totalSupplyAssets += assets.toUint128();
+      market[id].totalSupplyAssets -= assets.toUint128(); // MUTATION: flipped += to -=
for totalSupplyAssets in supply
```

withdraw mutations

#3  Removed totalSupplyAssets decrease in withdraw.

```
-      market[id].totalSupplyAssets -= assets.toUint128();
+      // market[id].totalSupplyAssets -= assets.toUint128(); // MUTATION: removed
totalSupplyAssets decrease in withdraw
```

borrow mutations

#4  Flipped += to -= for totalBorrowAssets in borrow.

```
-      market[id].totalBorrowAssets += assets.toUint128();  
+      market[id].totalBorrowAssets -= assets.toUint128(); // MUTATION: flipped += to -=  
for totalBorrowAssets in borrow
```

#5  Removed liquidity check in borrow.

```
-      require(market[id].totalBorrowAssets <= market[id].totalSupplyAssets,  
ErrorsLib.INSUFFICIENT_LIQUIDITY);  
+      // require(market[id].totalBorrowAssets <= market[id].totalSupplyAssets,  
ErrorsLib.INSUFFICIENT_LIQUIDITY); // MUTATION: removed liquidity check in borrow
```

#12  Zeroed totalBorrowAssets in borrow.

```
-      market[id].totalBorrowAssets += assets.toUint128();  
+      market[id].totalBorrowAssets = 0; // MUTATION: zeroed totalBorrowAssets in borrow
```

repay mutations

#6  Removed totalBorrowAssets decrease in repay.

```
-      market[id].totalBorrowAssets = UtilsLib.zeroFloorSub(market[id].totalBorrowAssets,  
assets).toUint128();  
+      // market[id].totalBorrowAssets =  
UtilsLib.zeroFloorSub(market[id].totalBorrowAssets, assets).toUint128(); // MUTATION:  
removed totalBorrowAssets decrease in repay
```

liquidate mutations

#7  Removed bad debt borrow assets decrease.

```
-      market[id].totalBorrowAssets -= badDebtAssets.toUint128();  
+      // market[id].totalBorrowAssets -= badDebtAssets.toUint128(); // MUTATION:  
removed bad debt borrow assets decrease
```

#8  Removed bad debt supply assets decrease.

```
-      market[id].totalSupplyAssets -= badDebtAssets.toUint128();  
+      // market[id].totalSupplyAssets -= badDebtAssets.toUint128(); // MUTATION:  
removed bad debt supply assets decrease
```

#9  Negated bad debt condition (== to !=).

```
-         if (position[id][borrower].collateral == 0) {
+         if (position[id][borrower].collateral != 0) { // MUTATION: negated bad debt
condition
```

_accrueInterest mutations

#10  Flipped interest to subtract from borrow.

```
-         market[id].totalBorrowAssets += interest.toUint128();
+         market[id].totalBorrowAssets -= interest.toUint128(); // MUTATION: flipped
interest to subtract from borrow
```

#11  Removed supply assets interest addition.

```
-         market[id].totalSupplyAssets += interest.toUint128();
+         // market[id].totalSupplyAssets += interest.toUint128(); // MUTATION: removed
supply assets interest addition
```

#13  Negated elapsed check (== to !=).

```
-         if (elapsed == 0) return;
+         if (elapsed != 0) return; // MUTATION: negated elapsed check in _accrueInterest
```

supplySharesSolvency

The `supplySharesSolvency` (invariant in [morpho_valid_state_many.spec](#)) asserts that total supply shares are at least as large as the sum of individual user supply shares. This prevents share dilution or phantom supply positions.

Result: [3/10 mutations caught](#) -- the invariant detects removal of total-level share updates but misses user-level mutations where the sum-of-parts relationship is not directly tracked.

Configuration: [valid_state_mutations_supplySharesSolvency.conf](#)

supply mutations

#1  Removed user supplyShares increase in supply.

```
-         position[id][onBehalf].supplyShares += shares;
+         // position[id][onBehalf].supplyShares += shares; // MUTATION: removed user
supplyShares increase in supply
```

#2  Removed totalSupplyShares increase in supply.

```
-         market[id].totalSupplyShares += shares.toUint128();
+         // market[id].totalSupplyShares += shares.toUint128(); // MUTATION: removed
totalSupplyShares increase in supply
```

#3  Flipped += to -= for user supplyShares in supply.

```
-         position[id][onBehalf].supplyShares += shares;
+         position[id][onBehalf].supplyShares -= shares; // MUTATION: flipped += to -= for
user supplyShares in supply
```

#9  Zeroed user supplyShares in supply.

```
-         position[id][onBehalf].supplyShares += shares;
+         position[id][onBehalf].supplyShares = 0; // MUTATION: zeroed user supplyShares in
supply
```

withdraw mutations

#4  Removed user supplyShares decrease in withdraw.

```
-         position[id][onBehalf].supplyShares -= shares;
+         // position[id][onBehalf].supplyShares -= shares; // MUTATION: removed user
supplyShares decrease in withdraw
```

#5  Removed totalSupplyShares decrease in withdraw.

```
-         market[id].totalSupplyShares -= shares.toUint128();
+         // market[id].totalSupplyShares -= shares.toUint128(); // MUTATION: removed
totalSupplyShares decrease in withdraw
```

#6  Flipped -= to += for totalSupplyShares in withdraw.

```
-         market[id].totalSupplyShares -= shares.toUint128();
+         market[id].totalSupplyShares += shares.toUint128(); // MUTATION: flipped -= to +=
for totalSupplyShares in withdraw
```

_accrueInterest mutations

#7  Removed feeRecipient supplyShares increase.

```
-         position[id][feeRecipient].supplyShares += feeShares;
+         // position[id][feeRecipient].supplyShares += feeShares; // MUTATION:
removed feeRecipient supplyShares increase
```

#8  Removed totalSupplyShares fee increase.

```
-         market[id].totalSupplyShares += feeShares.toUint128();
+         // market[id].totalSupplyShares += feeShares.toUint128(); // MUTATION:
removed totalSupplyShares fee increase
```

#10  Negated fee condition (!= to ==).

```
-         if (market[id].fee != 0) {
+         if (market[id].fee == 0) { // MUTATION: negated fee condition in
_accrueInterest
```

borrowSharesConservation

The `borrowSharesConservation` (rule in [morpho_high_level.spec](#)) asserts that the change in total borrow shares equals the change in the caller's borrow shares after a borrow operation. This is a conservation law -- no shares are created or destroyed outside the expected accounting.

Result: [7/10 mutations caught](#) -- the rule catches all share-tracking mutations (removals, sign flips, zeroing, doubling, off-by-one) but expectedly misses mutations to unrelated state (`totalBorrowAssets`) and preconditions (health check, computation condition).

Configuration: [high_level_mutations_borrowSharesConservation.conf](#)

borrow mutations

#1  Removed user borrowShares increase in borrow.

```
-         position[id][onBehalf].borrowShares += shares.toUint128();
+         // position[id][onBehalf].borrowShares += shares.toUint128(); // MUTATION: removed
user borrowShares increase in borrow
```

#2  Removed totalBorrowShares increase in borrow.

```
-         market[id].totalBorrowShares += shares.toUint128();
+         // market[id].totalBorrowShares += shares.toUint128(); // MUTATION: removed
totalBorrowShares increase in borrow
```

#3  Flipped += to -= for user borrowShares in borrow.

```
-         position[id][onBehalf].borrowShares += shares.toUint128();
+         position[id][onBehalf].borrowShares -= shares.toUint128(); // MUTATION: flipped +=
to -= for user borrowShares in borrow
```

#4  Flipped += to -= for totalBorrowShares in borrow.

```
-         market[id].totalBorrowShares += shares.toUint128();
+         market[id].totalBorrowShares -= shares.toUint128(); // MUTATION: flipped += to -=
for totalBorrowShares in borrow
```

#5  Zeroed user borrowShares in borrow.

```
-         position[id][onBehalf].borrowShares += shares.toUint128();
+         position[id][onBehalf].borrowShares = 0; // MUTATION: zeroed user borrowShares in
borrow
```

#6  Doubled user borrowShares in borrow.

```
-      position[id][onBehalf].borrowShares += shares.toUint128();
+      position[id][onBehalf].borrowShares += (shares * 2).toUint128(); // MUTATION:
doubled user borrowShares in borrow
```

#7  Off-by-one over for user borrowShares (+1).

```
-      position[id][onBehalf].borrowShares += shares.toUint128();
+      position[id][onBehalf].borrowShares += (shares + 1).toUint128(); // MUTATION: off-
by-one over for user borrowShares
```

#8  Removed health check in borrow.

```
-      require(_isHealthy(marketParams, id, onBehalf), ErrorsLib.INSUFFICIENT_COLLATERAL);
+      // require(_isHealthy(marketParams, id, onBehalf),
ErrorsLib.INSUFFICIENT_COLLATERAL); // MUTATION: removed health check in borrow
```

#9  Negated shares computation condition (assets > 0 to == 0).

```
-      if (assets > 0) shares = assets.toSharesUp(market[id].totalBorrowAssets,
market[id].totalBorrowShares);
+      if (assets == 0) shares = assets.toSharesUp(market[id].totalBorrowAssets,
market[id].totalBorrowShares); // MUTATION: negated assets > 0 to assets == 0
```

#10  Removed totalBorrowAssets increase in borrow.

```
-      market[id].totalBorrowAssets += assets.toUint128();
+      // market[id].totalBorrowAssets += assets.toUint128(); // MUTATION: removed
totalBorrowAssets increase in borrow
```

Overall mutation testing results: 14 caught / 33 total mutations (42% catch rate). The

`borrowSharesConservation` rule shows the strongest mutation detection at 70%, catching all share-tracking mutations. Survived mutations in `liquidityInvariant` and `supplySharesSolvency` are expected for mutations that affect only one side of the invariant inequality or modify state not directly tracked by the specific invariant.

Setup and Execution

The Certora Prover can be run either remotely (using Certora's cloud infrastructure) or locally (building from source). Both modes share the same initial setup steps.

Common Setup (Steps 1-5)

The instructions below are for Ubuntu 24.04. For step-by-step installation details refer to this setup [tutorial](#).

1. Install Java (tested with JDK 21)

```
sudo apt update
sudo apt install default-jre
java -version
```

2. Install [pipx](#) -- installs Python CLI tools in isolated environments, avoiding dependency conflicts

```
sudo apt install pipx
pipx ensurepath
```

3. Install Certora CLI. To match a specific prover version, pin it explicitly (e.g. `certora-cli==8.8.1`)

```
pipx install certora-cli
```

4. Install solc-select and the Solidity compiler version required by the project

```
pipx install solc-select
solc-select install 0.8.19
solc-select use 0.8.19
```

5. Create a versioned solc symlink for Certora. Configuration files reference the compiler as `solc0.8.19` (without dashes), but solc-select only creates a generic `solc` binary. Create the symlink so Certora can find it:

```
mkdir -p ~/.local/bin
ln -sf ~/.solc-select/artifacts/solc-0.8.19/solc-0.8.19 ~/.local/bin/solc0.8.19
```

Verify `~/.local/bin` is in your `PATH`. If not, add it:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```


Remote Execution

- Set up Certora key. You can get a free key through the Certora [Discord](#) or on their website. Once you have it, export it:

```
echo "export CERTORAKEY=<your_certora_api_key>" >> ~/.bashrc
```

Note: If a local prover is installed (see below), it takes priority. To force remote execution, add the `--server production` flag:

```
certoraRun certora/confs/valid_state_many.conf --server production
```

Local Execution

Follow the full build instructions in the [CertoraProver repository \(v8.8.1\)](#). Once the local prover is installed it takes priority over the remote cloud by default. Tested on Ubuntu 24.04.

- Install prerequisites

```
# JDK 19+
sudo apt install openjdk-21-jdk

# SMT solvers (z3 and cvc5 are required, others are optional)
# Download binaries and place them in PATH:
# z3: https://github.com/Z3Prover/z3/releases
# cvc5: https://github.com/cvc5/cvc5/releases

# LLVM tools
sudo apt install llvm

# Rust 1.81.0+
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
cargo install rustfilt

# Graphviz (optional, for visual reports)
sudo apt install graphviz
```

- Set up build output directory

```
export CERTORA="$HOME/CertoraProver/target/installed/"
mkdir -p "$CERTORA"
export PATH="$CERTORA:$PATH"
```

- Clone and build

```
git clone --recurse-submodules https://github.com/Certora/CertoraProver.git
cd CertoraProver
git checkout tags/8.8.1
./gradlew assemble
```

4. Verify installation with test example

```
certoraRun.py -h
cd Public/TestEVM/Counter
certoraRun counter.conf
```

Running Verification

Quick Runs

`variable_transitions`, `unit_tests`, `accrue_interest`, `reachability`, `reverts`, and `access_control` are lightweight and can be run as full conf files:

```
certoraRun certora/confs/variable_transitions.conf
certoraRun certora/confs/unit_tests.conf
certoraRun certora/confs/accrue_interest.conf
certoraRun certora/confs/reachability.conf
certoraRun certora/confs/reverts.conf
certoraRun certora/confs/access_control.conf
```

Note: `valid_state_one`, `valid_state_many`, `state_transitions`, and `high_level` may time out when running all rules at once. Run them per-rule using `--rule` as shown below.

Morpho Blue -- Valid State (one regime)

Runs every parametric method against the full valid-state invariant set under the one-market narrowing (scalar ghosts):

```
certoraRun certora/confs/valid_state_one.conf --rule feeBounded
certoraRun certora/confs/valid_state_one.conf --rule feeRequiresMarket
certoraRun certora/confs/valid_state_one.conf --rule lastUpdateBoundedByTimestamp
certoraRun certora/confs/valid_state_one.conf --rule lastUpdateMinBound
certoraRun certora/confs/valid_state_one.conf --rule liquidityInvariant
certoraRun certora/confs/valid_state_one.conf --rule supplySharesSolvency
certoraRun certora/confs/valid_state_one.conf --rule borrowSharesSolvency
certoraRun certora/confs/valid_state_one.conf --rule nonExistentMarketIsZero
certoraRun certora/confs/valid_state_one.conf --rule nonExistentMarketParamsAreZero
certoraRun certora/confs/valid_state_one.conf --rule nonExistentMarketPositionsZero
certoraRun certora/confs/valid_state_one.conf --rule marketIrmIsEnabled
certoraRun certora/confs/valid_state_one.conf --rule marketLltvIsEnabled
certoraRun certora/confs/valid_state_one.conf --rule enabledLltvBelowWad
certoraRun certora/confs/valid_state_one.conf --rule marketLltvBelowWad
certoraRun certora/confs/valid_state_one.conf --rule supplySharesRequiresMarket
certoraRun certora/confs/valid_state_one.conf --rule borrowSharesRequiresMarket
certoraRun certora/confs/valid_state_one.conf --rule collateralRequiresMarket
certoraRun certora/confs/valid_state_one.conf --rule supplyAssetsRequiresMarket
certoraRun certora/confs/valid_state_one.conf --rule supplySharesTotalRequiresMarket
certoraRun certora/confs/valid_state_one.conf --rule borrowAssetsRequiresMarket
certoraRun certora/confs/valid_state_one.conf --rule borrowSharesTotalRequiresMarket
certoraRun certora/confs/valid_state_one.conf --rule alwaysCollateralized
certoraRun certora/confs/valid_state_one.conf --rule zeroDoesNotAuthorize
```

Morpho Blue -- Valid State (many regime)

Runs every parametric method against the valid-state invariant set lifted to `forall MorphoHarness.Id id.` (per-id ghosts):

```
certoraRun certora/confs/valid_state_many.conf --rule feeBounded
certoraRun certora/confs/valid_state_many.conf --rule feeRequiresMarket
certoraRun certora/confs/valid_state_many.conf --rule lastUpdateBoundedByTimestamp
certoraRun certora/confs/valid_state_many.conf --rule lastUpdateMinBound
certoraRun certora/confs/valid_state_many.conf --rule liquidityInvariant
certoraRun certora/confs/valid_state_many.conf --rule supplySharesSolvency
certoraRun certora/confs/valid_state_many.conf --rule borrowSharesSolvency
certoraRun certora/confs/valid_state_many.conf --rule nonExistentMarketIsZero
certoraRun certora/confs/valid_state_many.conf --rule nonExistentMarketParamsAreZero
certoraRun certora/confs/valid_state_many.conf --rule nonExistentMarketPositionsZero
certoraRun certora/confs/valid_state_many.conf --rule marketIrmIsEnabled
certoraRun certora/confs/valid_state_many.conf --rule marketLltvIsEnabled
certoraRun certora/confs/valid_state_many.conf --rule enabledLltvBelowWad
certoraRun certora/confs/valid_state_many.conf --rule marketLltvBelowWad
certoraRun certora/confs/valid_state_many.conf --rule supplySharesRequiresMarket
certoraRun certora/confs/valid_state_many.conf --rule borrowSharesRequiresMarket
certoraRun certora/confs/valid_state_many.conf --rule collateralRequiresMarket
certoraRun certora/confs/valid_state_many.conf --rule supplyAssetsRequiresMarket
certoraRun certora/confs/valid_state_many.conf --rule supplySharesTotalRequiresMarket
certoraRun certora/confs/valid_state_many.conf --rule borrowAssetsRequiresMarket
certoraRun certora/confs/valid_state_many.conf --rule borrowSharesTotalRequiresMarket
certoraRun certora/confs/valid_state_many.conf --rule alwaysCollateralized
certoraRun certora/confs/valid_state_many.conf --rule zeroDoesNotAuthorize
```

State Transitions

```
certoraRun certora/confs/state_transitions.conf --rule marketCreationAtomicity
certoraRun certora/confs/state_transitions.conf --rule accountingChangesRefreshTimestamp
certoraRun certora/confs/state_transitions.conf --rule
lastUpdateChangeRequiresAccountingOrFeeChange
certoraRun certora/confs/state_transitions.conf --rule
userSupplySharesIncreaseImpliesTotalIncrease
certoraRun certora/confs/state_transitions.conf --rule
userSupplySharesDecreaseImpliesTotalDecrease
certoraRun certora/confs/state_transitions.conf --rule
userBorrowSharesIncreaseImpliesTotalIncrease
certoraRun certora/confs/state_transitions.conf --rule
userBorrowSharesDecreaseImpliesTotalDecrease
certoraRun certora/confs/state_transitions.conf --rule
totalSupplySharesIncreaseImpliesAssetsIncrease
certoraRun certora/confs/state_transitions.conf --rule
totalSupplySharesDecreaseImpliesAssetsDecrease
certoraRun certora/confs/state_transitions.conf --rule
totalBorrowSharesIncreaseImpliesAssetsIncrease
certoraRun certora/confs/state_transitions.conf --rule interestAccrualSymmetry
certoraRun certora/confs/state_transitions.conf --rule
collateralAndBorrowDecreaseImpliesTotalBorrowSharesDecrease
certoraRun certora/confs/state_transitions.conf --rule supplyIncreasesContractBalance
```

High-Level

```
certoraRun certora/confs/high_level.conf --rule supplySharesConservation
certoraRun certora/confs/high_level.conf --rule withdrawSharesConservation
certoraRun certora/confs/high_level.conf --rule borrowSharesConservation
certoraRun certora/confs/high_level.conf --rule repaySharesConservation
certoraRun certora/confs/high_level.conf --rule supplyAssetConservation
certoraRun certora/confs/high_level.conf --rule supplyWithdrawNoProfit
certoraRun certora/confs/high_level.conf --rule borrowRepayNoProfit
certoraRun certora/confs/high_level.conf --rule accrueInterestIncreasesSupplyAssets
certoraRun certora/confs/high_level.conf --rule accrueInterestIncreasesBorrowAssets
certoraRun certora/confs/high_level.conf --rule accrueInterestEqualDelta
certoraRun certora/confs/high_level.conf --rule liquidationPreservesLiquidityRelation
certoraRun certora/confs/high_level.conf --rule liquidationBorrowSharesConservation
certoraRun certora/confs/high_level.conf --rule supplyCollateralDoesNotChangeTotals
certoraRun certora/confs/high_level.conf --rule nonLiquidationPreservesCollateralization
```

Running Mutation Testing

Run the mutation test suite for each tested property:

```
certoraMutate certora/mutations/liquidityInvariant.conf
certoraMutate certora/mutations/supplySharesSolvency.conf
certoraMutate certora/mutations/borrowSharesConservation.conf
```

Creating Mutations for Other Invariants

1. **Create the mutations directory:** `mkdir -p certora/mutations/<invariantName>`
2. **Introduce a mutation** into the source contract (e.g., comment out a line, flip a condition)
3. **Run the helper script** to snapshot the mutation:

```
./certora/mutations/add_mutation.sh <invariantName> src/Morpho.sol
```

This auto-numbers the mutation file (e.g., `1.sol`, `2.sol`, ...) and embeds the diff block. The original source is automatically restored via `git restore`.

4. **Create a conf file** (copy an existing one and add the `mutations` section pointing to your new directory)
5. **Run** `certoraMutate` with your new conf file